CONTROL DE CAMBIOS - REFACTORIZACIÓN PRINCIPIOS SOLID

CC-02: Violación del Principio de Responsabilidad Única (SRP)

Campo	Valor
ID	CC-02
Archivo / Clase Modificada	`src/modules/auth/auth.controller.ts` - Métodos `register()` y `login()`
Fallo SOLID / Antipatrón	Violación del Principio de Responsabilidad Única (SRP - Single Responsibility Principle)
Descripción del Problema	El controlador tiene **dos responsabilidades conflictivas**: (1) Orquestar la respuesta HTTP y (2) Validar/limpiar datos entrantes con `registerSchema.parse()` y `loginSchema.parse()`. Si la estructura de validación cambia, el controlador se ve afectado. Esto mezcla capas de la arquitectura (validación en el nivel de controlador en lugar de middleware), dificulta testing aislado del controlador y tensa el principio SRP que dice "Una clase debe tener una única razón para cambiar".
Mejora Aplicada	Se implementó un **Middleware de Validación Genérico** (`validateRequest`) que encapsula toda la lógica Zod fuera del controlador. Cada ruta inyecta `validateRequest(schema)` como middleware previo. El controlador recibe datos ya validados en `req.validatedData`, confiando que son perfectos y dedicándose exclusivamente a orquestación HTTP. Nuevos schemas o cambios de validación afectan solo el middleware y las rutas, nunca el controlador.
Severidad	**M (Media)** - Mezcla capas de arquitectura, dificulta testing aislado del controlador, aumenta acoplamiento.

Evidencia ANTES (Violación de SRP):

```
// ❌ Controlador hace DOS cosas: orquestar HTTP + validar datos
export class AuthController {
  async register(req: Request, res: Response): Promise<void> {
    // Responsabilidad 1: Validación (debería estar en middleware)
    const input = registerSchema.parse(req.body);

    // Responsabilidad 2: Orquestación HTTP
    const user = await this.service.register(input);
    res.status(201).json({ user });
  }

  async login(req: Request, res: Response): Promise<void> {
    // Responsabilidad 1: Validación (debería estar en middleware)
    const input = loginSchema.parse(req.body);

    // Responsabilidad 2: Orquestación HTTP
    const session = await this.service.login(input);
```

```
    res.json(session);
  }
}
```

Problemas:

- Si cambia `registerSchema` (ej: agregar campo, cambiar validación), hay que modificar el controlador
- Si cambia el formato de error de validación, hay que modificar el controlador
- Difícil hacer unit-test del controlador sin mockear Zod
- Mezcla responsabilidades: negocio HTTP + validación de datos
- Violación de SRP: "Una clase debe tener una única razón para cambiar"

Evidencia DESPUÉS (Implementación de Middleware de Validación + SRP):

1. Middleware Genérico de Validación:

```
// src/shared/middleware/validate-request.middleware.ts
export function validateRequest(schema: ZodSchema) {
  return async (req: Request, res: Response, next: NextFunction) => {
    try {
      // Validación ocurre aquí, no en el controlador
      const validated = schema.parse(req.body);

      // Adjuntar datos validados al request
      req.validatedData = validated;

      next();
    } catch (error: any) {
      // Manejo de errores de validación centralizado
      if (error.errors && Array.isArray(error.errors)) {
        return res.status(400).json({
          error: 'Validation failed',
          details: error.errors.map((e: any) => ({
            field: e.path.join('.'),
            message: e.message,
          })),
        });
      }

      res.status(400).json({
        error: 'Validation failed',
        message: error.message,
      });
    }
  };
}
```

Beneficio: Validación reutilizable, centralizada, testeable por separado.

2. Controlador Refactorizado (SRP Compliant):

```
// src/modules/auth/auth.controller.ts
export class AuthController {
```

```
  async register(req: Request, res: Response): Promise<void> {
    // ✅ Datos ya validados por middleware
    const input = req.validatedData as RegisterInput;

    // ✅ Solo orquestación HTTP (una única responsabilidad)
    const user = await this.service.register(input);
    res.status(201).json({ user });
  }

  async login(req: Request, res: Response): Promise<void> {
    // ✅ Datos ya validados por middleware
    const input = req.validatedData as LoginInput;

    // ✅ Solo orquestación HTTP (una única responsabilidad)
    const session = await this.service.login(input);
    res.json(session);
  }
}
```

Beneficio: Controlador tiene UNA responsabilidad: orquestar HTTP.

3. Rutas Inyectan Middleware de Validación:

```
// src/modules/auth/auth.routes.ts
export function createAuthRouter(): Router {
  const repository = new AuthRepository();
  const service = new AuthService(repository);
  const controller = new AuthController(service);

  const router = Router();

  // ✅ Validación en middleware, no en controlador
  router.post(
    '/register',
    validateRequest(registerSchema),  // ← Middleware de validación
    asyncHandler(controller.register.bind(controller))
  );

  router.post(
    '/login',
    validateRequest(loginSchema),     // ← Middleware de validación
    asyncHandler(controller.login.bind(controller))
  );

  return router;
}
```

Beneficio: Separación clara de responsabilidades. Rutas = composición, Controlador = orquestación.

Beneficios de la Mejora:

Aspecto	Antes	Después
SRP	❌ 2 responsabilidades	✅ 1 responsabilidad
Cambios en validación	❌ Modificar controlador	✅ Modificar solo middleware/rutas
Testabilidad	❌ Difícil aislar lógica HTTP	✅ Fácil testear middleware y controlador por separado
Reutilización	❌ Validación solo en auth	✅ Middleware reutilizable en todos los módulos
Mantenibilidad	❌ Lógica de validación dispersa	✅ Validación centralizada en middleware
Acoplamiento	❌ Controlador → Zod	✅ Controlador → Express Request (genérico)

Patrón Aplicado:

- **Middleware Pattern**: Middleware de validación genérico antes del controlador
- **Separation of Concerns**: Validación (middleware) vs. Orquestación (controlador)
- **Chain of Responsibility**: Cadena middleware → controlador → servicio

---

Cómo Reutilizar en Otros Módulos:

```
// En cualquier ruta
router.post(
  '/create-work-order',
  validateRequest(createWorkOrderSchema),  // ← Reutilizable
  asyncHandler(workOrderController.create.bind(workOrderController))
);

router.put(
  '/update-product/:id',
  validateRequest(updateProductSchema),    // ← Reutilizable
  asyncHandler(productController.update.bind(productController))
);
```

✅ **Sin modificar validación de ningún controlador existente**

Resumen de Cambios

Antes	Después
Validación dentro del controlador	Validación en middleware previo
Controlador = orquestación + validación	Controlador = solo orquestación
Cambios en schemas afectan controlador	Cambios en schemas solo afectan middleware
Validación no reutilizable	Validación reutilizable en todas las rutas
Difícil testear controlador aislado	Fácil testear middleware y controlador por separado

---

Archivos Modificados y Creados

✅ CREADOS:

1. `src/shared/middleware/validate-request.middleware.ts` ✅
   - Middleware de validación genérico para cualquier schema Zod
   - Manejo centralizado de errores de validación
   - Adjunta datos validados a `req.validatedData`

✅ MODIFICADOS:

1. `src/modules/auth/auth.controller.ts` ✅
   - **ANTES**: `const input = registerSchema.parse(req.body);`
   - **DESPUÉS**: `const input = req.validatedData as RegisterInput;`
   - Eliminada importación de schemas
   - Controlador ahora confía en que datos son válidos
2. `src/modules/auth/auth.routes.ts` ✅
   - Agrega `validateRequest(registerSchema)` en ruta `/register`
   - Agrega `validateRequest(loginSchema)` en ruta `/login`
   - Importa middleware de validación
   - Middleware ejecuta ANTES que el controlador

Estado: ✅ **COMPLETADO Build Status**: ✅ TypeScript + npm run build → SUCCESS **Principio Implementado**: SRP = ✅ CUMPLIDO (controlador tiene una única responsabilidad) **Patrón Aplicado**: Middleware de Validación + Separation of Concerns **Resultado**: Validación separada de orquestación HTTP, reutilizable y fácil de mantener

---

EXTENSIÓN A OTROS MÓDULOS (Próximas iteraciones)

El middleware `validateRequest` es genérico y puede reutilizarse en **todos los módulos** sin modificaciones. Ejemplo en `work-orders`:

Refactorización de work-orders.controller.ts:

ANTES (Violación SRP):

```
async create(req: Request, res: Response): Promise<void> {
  const input = createWorkOrderSchema.parse(req.body);  // ❌ Validación aquí
  const order = await this.service.create(input, 'ACEPTADA');
  res.status(201).json(order);
}
```

DESPUÉS (SRP Compliant):

```
async create(req: Request, res: Response): Promise<void> {
  const input = req.validatedData as CreateWorkOrderInput;  // ✅ Datos validados
  const order = await this.service.create(input, 'ACEPTADA');
  res.status(201).json(order);
}
```

En work-orders.routes.ts:

```
router.post(
  '/',
  validateRequest(createWorkOrderSchema),  // ← Middleware reutilizable
  asyncHandler(controller.create.bind(controller))
);
```

Módulos Afectados (Mismo patrón SRP violado):

- `customers` (2 métodos: create, update)
- `technicians` (3 métodos: register, update, setActive)
- `suppliers` (2 métodos: create, update)
- `services` (2 métodos: create, update)
- `products` (3 métodos: create, update, registerMovement)
- `parts` (2 métodos: create, update)
- `appointments` (1 método: create)
- `diagnostics` (2 métodos: create, update)
- `warranties` (1 método: create)
- `technician-availability` (1 método: update)
- `purchase-requests` (2 métodos: create, updateStatus)
- `work-orders` (5 métodos: create, createAsClient, transitionStatus, registerExit, createActivity)

Total: ~31 métodos que podrían beneficiarse de esta refactorización.

Plan de Extensión (No ejecutado en esta iteración):

1. Todos los módulos seguirían el mismo patrón
2. Controladores NUNCA importarían Zod schemas directamente
3. Rutas inyectarían `validateRequest(schema)` antes del controlador
4. Cambios en validación solo afectan middleware y rutas, nunca controladores

Beneficio: Una vez aplicado, SRP en validación = ✅ CUMPLIDO para todo el backend.

Nota Final: Este documento (CC-02) documenta la **primera implementación** del patrón de validación SRP en el módulo `auth`. La misma solución puede clonarse a los otros 12 módulos siguiendo el mismo patrón, sin necesidad de crear nuevos middleware o interfaces.